



```
fn main() -> i32 {  
    println("Hello, Amiga!")  
    return 0  
}
```

NOVUS

Programmer's Reference Guide

New code for classic machines

Version 0.1 | December 2025

For Amiga 68020+ | AmigaOS 2.0+

Novus Programmer's Reference Guide

Version 0.1 — December 2025

Copyright © 2025, 2026 Barry Walker.
All Rights Reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Novus is an open source project.
Visit <https://github.com/barryw/Novus> for source code and updates.

Amiga is a trademark of Amiga Corporation.
All other trademarks are the property of their respective owners.

Typeset in Latin Modern using L^AT_EX.

Contents

Foreword	iii
1 Introduction	1
1.1 What is Novus?	1
1.1.1 Key Characteristics	1
1.2 Why Novus Exists	1
1.2.1 The Limits of C	1
1.2.2 Modern Solutions, Amiga Constraints	2
1.3 Design Philosophy	2
1.3.1 Explicit Over Implicit	2
1.3.2 Predictable Performance	3
1.3.3 Respect the Machine	3
1.3.4 Fail Fast, Fail Clearly	3
1.4 Target Audience	3
1.5 How to Read This Guide	3
2 Getting Started	5
2.1 Prerequisites	5
2.2 Installation	5
2.2.1 Building from Source	5
2.2.2 Verifying Installation	5
2.3 Your First Program	6
2.3.1 Compiling	6
2.3.2 Running on the Amiga	6
2.4 Understanding the Toolchain	6
2.4.1 Compiler Commands	7
2.4.2 Compile Options	7
2.4.3 Debug vs Release Builds	7
2.5 Project Structure	8
2.5.1 The project.toml File	8
2.6 The Standard Library	9
2.7 Running Tests	9
2.7.1 Test Options	10
2.8 Next Steps	10
3 Language Basics	11
3.1 Comments	11
3.1.1 Single-Line Comments	11
3.1.2 Multi-Line Comments	11
3.2 Variables and Mutability	11
3.2.1 Mutable Variables with <code>var</code>	11
3.2.2 Immutable Bindings with <code>let</code>	12
3.2.3 Compile-Time Constants with <code>const</code>	12
3.2.4 Comparison	12
3.2.5 Type Annotations	12

3.3	Primitive Types	12
3.3.1	Integer Types	13
3.3.2	Floating-Point Types	13
3.3.3	Fixed-Point Types	13
3.3.4	Boolean Type	13
3.4	Literals	13
3.4.1	Integer Literals	13
3.4.2	Floating-Point Literals	14
3.4.3	String Literals	14
3.4.4	F-Strings (Formatted Strings)	15
3.4.5	Character Literals	15
3.5	Operators	15
3.5.1	Arithmetic Operators	15
3.5.2	Comparison Operators	16
3.5.3	Logical Operators	16
3.5.4	Bitwise Operators	16
3.5.5	Compound Assignment Operators	17
3.5.6	Increment and Decrement Operators	17
3.5.7	Range Operators	18
3.6	Type Conversions	18
3.6.1	C-Style Casts	18
3.6.2	The <code>as</code> Keyword	18
3.7	References and Pointers	18
3.7.1	Immutable References	18
3.7.2	Mutable References	19
3.7.3	Raw Pointers	19
3.7.4	Dereferencing	19
3.8	Arrays	19
3.8.1	Fixed-Size Arrays	19
3.8.2	Array Literals	19
3.8.3	Repeat Syntax	20
3.8.4	Indexing	20
3.9	Tuples	20
3.9.1	Tuple Types	20
3.9.2	Tuple Literals	20
3.9.3	Deconstructing	20
3.9.4	Returning Multiple Values	20
3.10	Expressions and Statements	21
3.10.1	Expression vs Statement	21
3.10.2	Block Expressions	21
3.11	Summary	21

Foreword

The Amiga was ahead of its time. In 1985, while other personal computers offered beeps and static screens, the Amiga delivered pre-emptive multitasking, dedicated graphics and audio hardware, and a sophisticated operating system that wouldn't be matched by mainstream competitors for nearly a decade.

For many of us, the Amiga wasn't just a computer—it was where we learned to code, to create, to push hardware to its limits. The demoscene flourished. Games achieved visual and audio fidelity that seemed impossible. A generation of programmers cut their teeth on 68000 assembly, crafting tight loops that squeezed every cycle from the machine.

Then the platform faded from the mainstream. But it never truly died.

Today, a vibrant community keeps the Amiga alive. Original hardware is lovingly maintained and expanded. FPGA recreations achieve cycle-perfect accuracy. Emulation lets new generations experience what made these machines special. And remarkably, people still write software for them—not out of obligation, but out of genuine passion.

Yet the tools available to modern Amiga developers are largely the same ones we used thirty years ago. C compilers like VBCC and GCC produce excellent code, but the languages themselves haven't evolved. The patterns and safety mechanisms that modern systems programming takes for granted—sum types, pattern matching, resource management without garbage collection—remain out of reach.

Novus aims to change that.

This is not an attempt to modernize the Amiga by hiding what makes it special. Quite the opposite. Novus is designed to *embrace* the Amiga's architecture: its custom chips, its message-passing OS, its elegant hardware. The goal is to give developers modern ergonomics and safety while preserving—even enhancing—the direct hardware access that makes Amiga programming rewarding.

Whether you're a veteran who remembers waiting for Workbench to load from floppy, or a newcomer curious about this legendary platform, we hope Novus helps you write code that's safer, clearer, and more enjoyable—without sacrificing an ounce of the control that Amiga development demands.

New code for classic machines. Welcome to Novus.

December 2025

Chapter 1

Introduction

1.1 What is Novus?

Novus is a systems programming language designed specifically for the Amiga 68k platform. It transpiles to C99, which VBCC then compiles to native 68020+ machine code. The result is executables that integrate deeply with AmigaOS and run on real hardware and accurate emulators alike.

The name comes from the Latin word for “new”—reflecting the project’s mission to bring modern language design to classic machines. Novus is not a port of an existing language, nor is it a lowest-common-denominator cross-platform tool. Every design decision is made with the Amiga in mind.

1.1.1 Key Characteristics

Native Performance Novus transpiles to C99, leveraging VBCC’s mature 68k code generator to produce Amiga executables. There is no interpreter, no virtual machine, no runtime overhead beyond what you explicitly request.

Modern Safety The type system catches errors at compile time. `Result` and `Option` types make error handling explicit. Resource cleanup is deterministic via `defer` blocks. You get safety without garbage collection.

Direct Hardware Access Novus doesn’t hide the Amiga’s hardware behind abstractions you can’t escape. Custom chip registers, Exec library calls, copper lists, blitter operations—all are first-class citizens with typed, safe interfaces.

Readable Output The generated C code is clean and readable. When you need to understand what the compiler is doing—and on the Amiga, you often do—the output won’t fight you. You can also emit assembly via `-emit-asm` to see exactly what VBCC produces.

Amiga-Native Async The `async/await` model maps directly to AmigaOS primitives: Exec signals and message ports. No hidden threads, no runtime scheduler—just the OS facilities the Amiga provides.

1.2 Why Novus Exists

The Amiga platform has capable C compilers. VBCC, in particular, generates excellent code and remains actively maintained. So why create a new language?

1.2.1 The Limits of C

C was revolutionary in 1972. It remains useful today precisely because it’s minimal—a portable assembler that gets out of your way. But that minimalism has costs:

- **No sum types.** Representing “this value is either an error or a result” requires conventions that the compiler can’t enforce.
- **No built-in error handling.** Return codes are easily ignored. Exceptions don’t exist. Every function call is an opportunity for silent failure.
- **Manual resource management.** Matching every `AllocMem` with a `FreeMem`, every `OpenLibrary` with a `CloseLibrary`, every `Lock` with an `UnLock`—across all control flow paths, including error cases.
- **Primitive module system.** Header files and `#include` are textual substitution. Name collisions, include order dependencies, and compile time explosion are facts of life.
- **No metaprogramming.** The preprocessor is powerful but crude. Anything beyond simple macros quickly becomes write-only code.

These aren’t theoretical concerns. They’re the source of real bugs in real Amiga software—memory leaks, use-after-free, null pointer crashes, resource exhaustion from unclosed libraries.

1.2.2 Modern Solutions, Amiga Constraints

Languages like Rust address these problems elegantly. But Rust targets modern systems with megabytes of RAM, gigahertz CPUs, and operating systems that provide virtual memory and threads. Its runtime assumptions don’t map to the Amiga.

Novus takes a different approach: provide modern ergonomics within Amiga constraints.

- `Result[T, E]` and `Option[T]` are zero-cost at runtime—they’re just tagged unions that the compiler understands.
- `defer` blocks compile to straightforward cleanup code inserted at scope exit. No hidden allocations, no unwinding tables.
- Pattern matching compiles to efficient jump tables or comparison chains as appropriate.
- The module system is compile-time only. No runtime symbol resolution, no dynamic linking overhead.
- Fixed-point math uses inline 68020+ instructions, not floating-point emulation.

The result is a language that *feels* modern but *runs* like hand-tuned assembly.

1.3 Design Philosophy

Novus follows several core principles that inform every language decision:

1.3.1 Explicit Over Implicit

If something allocates memory, the code should say so. If something can fail, the type should reflect it. If a function has side effects, they should be visible at the call site.

This doesn’t mean verbose—Novus has type inference, sensible defaults, and concise syntax. But it means no hidden costs, no action at a distance, no “magic” that obscures what the machine is actually doing.

1.3.2 Predictable Performance

On a 7 MHz 68000, every cycle matters. On a 50 MHz 68060, you have more headroom—but you still can’t afford hidden allocations in a tight loop or cache-hostile memory access patterns.

Novus gives you control. The compiler won’t secretly box values, won’t insert invisible runtime checks in release builds, won’t generate code that’s impossible to reason about.

1.3.3 Respect the Machine

The Amiga isn’t a Unix workstation. It has custom chips that do things no other hardware can do. Its operating system uses message passing and signals, not files and processes. Its memory model is flat and physical.

Novus doesn’t pretend otherwise. The standard library wraps AmigaOS idiomatically—`Result` types for operations that can fail, handles for resources that need cleanup—but it doesn’t try to be POSIX. Amiga code should look like Amiga code.

1.3.4 Fail Fast, Fail Clearly

When something goes wrong, you should know immediately and know exactly what happened. Compile-time errors are better than runtime errors. Runtime errors with clear messages are better than silent corruption.

In debug builds, Novus checks array bounds, validates pointers, and panics with meaningful diagnostics. In release builds, those checks can be removed—but the type system still prevents the errors that checks would catch.

1.4 Target Audience

This guide assumes you’re a working programmer. You should be comfortable with:

- Systems programming concepts: pointers, memory allocation, stack vs. heap
- Basic 68k architecture: registers, addressing modes, the supervisor/user split
- AmigaOS fundamentals: libraries, Exec, DOS, the message-passing model

Prior experience with Rust, Swift, or other modern systems languages will make Novus feel familiar, but it’s not required. If you’ve written C for the Amiga, you have all the background you need.

1.5 How to Read This Guide

The guide is organized in layers:

Part I: Language Fundamentals covers the core language—syntax, types, control flow, functions, and modules. Read this first.

Part II: Standard Library documents the `std` packages: collections, strings, memory management, and Amiga OS bindings.

Part III: Advanced Topics covers async programming, inline assembly, FFI with existing C code, and performance optimization.

Appendices provide quick references, grammar summaries, and migration guides from C.

Code examples are designed to be complete and runnable. When a snippet requires context, we provide it. When we omit boilerplate, we say so.

Let’s begin.

Chapter 2

Getting Started

“The journey of a thousand miles begins with a single step.”

—Lao Tzu

This chapter walks you through installing the Novus compiler, writing your first program, and understanding the compilation pipeline. By the end, you’ll have a working executable running on your Amiga (or emulator).

2.1 Prerequisites

Before installing Novus, ensure you have:

- **.NET 9.0 or later** — The Novus compiler is written in C# and requires the .NET runtime. Download from <https://dotnet.microsoft.com>
- **VBCC toolchain** — Novus uses VBCC’s assembler (`vasmm68k_mot`) and linker (`vlink`) to produce Amiga executables. The toolchain is vendored in the repository under `vendor/vbcc`, or you can specify a custom installation via the `-vbcc-path` option.
- **An Amiga or emulator** — To run your compiled programs. FS-UAE, WinUAE, or real hardware all work. A 68020+ configuration with AmigaOS 2.0 or later is required.

2.2 Installation

2.2.1 Building from Source

Clone the repository and build:

```
git clone https://github.com/your-repo/novus.git
cd novus
dotnet build
```

The compiler will be available at `Novus/bin/Debug/net9.0/novus`.
For a release build with optimizations:

```
dotnet build -c Release
```

2.2.2 Verifying Installation

Check that the compiler runs:

```
./Novus/bin/Debug/net9.0/novus --help
```

You should see the available commands and options.

2.3 Your First Program

Create a file named `hello.novus`:

```
from std::io::file import println

fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

Let's break this down:

- `from std::io::file import println` — Imports the `println` function from the standard library. Unlike some languages, Novus requires explicit imports.
- `fn main() -> i32` — Declares the entry point. All Novus programs must have a `main` function that returns an `i32` (the exit code).
- `println("Hello, Amiga!")` — Prints the message to standard output.
- `return 0` — Returns success (zero) to the operating system.

2.3.1 Compiling

Compile to an Amiga executable:

```
novus compile hello.novus -o hello
```

This produces an executable named `hello` in Amiga HUNK format. Without the `-o` flag, the output would default to `a.out`.

2.3.2 Running on the Amiga

Transfer the executable to your Amiga (via shared folder, network, or disk image) and run it from the Shell:

```
1.Workbench:> hello
Hello, Amiga!
```

Congratulations—you've run your first Novus program!

2.4 Understanding the Toolchain

The Novus compilation pipeline has several stages:

1. **Parsing** — Source code is parsed into an Abstract Syntax Tree (AST)
2. **Semantic Analysis** — Types are checked, names are resolved, and the Intermediate Representation (IR) is built
3. **C Code Generation** — The IR is lowered to C99 code targeting VBCC
4. **Compilation** — VBCC's C compiler (`vc`) compiles the C code directly to 68k object files

5. **Assembly** — Additional assembly files (runtime, library stubs) are assembled with `vasm`
6. **Linking** — `vlink` combines all object files with the runtime and produces the final executable

This pipeline means you get the benefits of VBCC’s mature 68k code generation while writing modern, safe Novus code.

2.4.1 Compiler Commands

Novus provides several commands:

novus compile Compile a single source file to an executable

novus build Build a project defined by `project.toml`

novus test Discover and compile tests

novus new Create a new project from a template

novus fmt Format source code

novus clean Remove cached build artifacts

2.4.2 Compile Options

Common options for `novus compile`:

-o, -output Output file name (default: `a.out`)

-cpu Target CPU: 68020, 68030, 68040, 68060, or `auto` (default: `auto`)

-fpu FPU mode: `none`, 68881, 68882, 68040, 68060, or `auto` (default: `auto`)

-release Enable optimizations and disable debug checks

-O Optimization level (0–2)

-emit-asm Emit assembly output only; do not assemble or link

-emit-ir Emit IR to stdout for debugging

-v, -verbose Show detailed compilation progress

The `auto` CPU setting produces a “fat binary” that detects the CPU at runtime and uses optimized code paths accordingly.

2.4.3 Debug vs Release Builds

By default, Novus compiles in debug mode:

- Array bounds checking is enabled
- Debug symbols are included
- Optimizations are minimal

For production builds, use `-release`:

```
novus compile hello.novus -o hello --release
```

Release builds are smaller and faster, but runtime errors provide less diagnostic information.

Safety Levels

For finer control, use `-safety-level`:

Level 0 Unsafe — no runtime checks (use with `-unsafe` flag)

Level 1 Basic — minimal checks

Level 2 Full — standard safety checks (default for debug)

Level 3 Paranoid — maximum checking

2.5 Project Structure

For anything beyond a single file, create a project with `novus new`:

```
novus new myproject --template cli
cd myproject
```

This creates a standard project structure:

```
myproject/
  project.toml      # Project configuration
  src/
    main.novus     # Entry point
```

2.5.1 The project.toml File

The `project.toml` file defines project metadata and build settings:

```
[package]
name = "myproject"
version = "0.1.0"

[build]
target_cpu = "68020"
fpu = "none"
```

Available build settings include:

target_cpu Target CPU (68020, 68030, 68040, 68060, or auto)

fpu FPU mode

chipset Target chipset (ocs, ecs, aga, or auto)

optimization_level Optimization level (0–2)

output Output file name

Build the project with:

```
novus build
```


2.6 The Standard Library

Novus includes a standard library providing common functionality:

std::core Fundamental types: `Option`, `Result`, and core traits

std::collections Data structures: `Vec`, `HashMap`, `HashSet`, `VecDeque`, `RingBuffer`, `SlotMap`, and more

std::strings String handling: `String`, `Str`, `StringBuilder`

std::memory Memory management: `MemoryBlock`, chip memory pools, slices

std::math Fixed-point arithmetic, trigonometry, vectors, easing functions

std::io Input/output: `println`, file operations

std::ffi AmigaOS bindings: `Exec`, `DOS`, `Graphics`, `Intuition`, and more

Import modules with `from`:

```
from std::collections::vec import Vec
from std::io::file import println

fn main() -> i32 {
    var names: Vec<&str> = Vec::new()
    names.push("Amiga")
    names.push("Novus")

    println("Count: {}", names.len())
    return 0
}
```

2.7 Running Tests

Novus has built-in test support. Mark test functions with the `@test` attribute:

```
from std::test::test import expect_eq

@test
fn test_addition() {
    expect_eq(2 + 2, 4, "basic addition")
}

@test
fn test_multiplication() {
    expect_eq(3 * 7, 21, "basic multiplication")
}
```

The `expect_eq` function uses overloading to work with any comparable type—no need for type-specific variants like `expect_eq_i32`.

Compile the test runner with:

```
novus test mytest.novus
```

This discovers all `@test` functions and compiles a test executable. The output tells you where the executable was created:

```
Test runner built successfully!
Output: /path/to/tests
Tests: 2 active, 0 skipped

Copy to Amiga and run to execute tests.
```

Important: The test command builds the test runner but does not execute it. You must transfer the executable to your Amiga and run it there to see test results.

2.7.1 Test Options

The `novus test` command supports several options:

- `-filter` Run only tests matching a pattern
- `-list` List discovered tests without building
- `-b`, `-benchmark` Include timing information in output
- `-output-dir` Specify where to write the test executable

2.8 Next Steps

You now have a working Novus installation and understand the basic workflow. The following chapters cover:

- **Chapter 3: Language Basics** — Variables, types, and expressions
- **Chapter 4: Types** — The type system in depth
- **Chapter 5: Control Flow** — Conditionals, loops, and pattern matching

When you're ready, turn the page and start learning the language itself.

Chapter 3

Language Basics

“Simplicity is prerequisite for reliability.”

—Edsger W. Dijkstra

This chapter covers the fundamental building blocks of Novus: how to declare variables, what types are available, how literals work, and how expressions combine values. By the end, you’ll understand the core syntax and be ready to write meaningful programs.

3.1 Comments

Novus supports two comment styles:

3.1.1 Single-Line Comments

Use `//` for comments that extend to the end of the line:

```
// This is a single-line comment
let x = 42 // Comments can appear after code
```

3.1.2 Multi-Line Comments

Use `/* */` for comments spanning multiple lines:

```
/*
 * This is a multi-line comment.
 * Useful for longer explanations.
 */
let result = compute()
```

Multi-line comments do not nest. The first `*/` closes the comment, regardless of how many `/*` appeared inside.

3.2 Variables and Mutability

Novus provides three keywords for declaring bindings:

3.2.1 Mutable Variables with `var`

Variables declared with `var` can be reassigned:

```
var x = 10
x = 20 // OK - can reassign
x = x + 5 // OK
```

Use `var` for counters, accumulators, temporary values, and any data that changes during execution.

3.2.2 Immutable Bindings with `let`

Bindings declared with `let` cannot be reassigned:

```
let x = 10
x = 20 // ERROR - cannot reassign
```

Immutable bindings make code intent clearer and allow the compiler to optimize more aggressively. Use `let` by default; switch to `var` when you need mutability.

Note that immutability applies to the binding, not necessarily to the data it refers to. For example, a `let` binding to a mutable reference (`&var T`) still allows modifying the referenced data.

3.2.3 Compile-Time Constants with `const`

Constants declared with `const` are evaluated at compile time:

```
pub const SCREEN_WIDTH: u32 = 320
pub const SCREEN_HEIGHT: u32 = 200
pub const SCREEN_SIZE: u32 = SCREEN_WIDTH * SCREEN_HEIGHT
```

Constants must be literals or constant expressions—values that the compiler can evaluate without executing code. They cannot call functions or use runtime values.

Constants are typically declared at module scope with `pub` visibility.

3.2.4 Comparison

Keyword	Reassignable	Evaluation	Typical Scope
<code>var</code>	Yes	Runtime	Function/Block
<code>let</code>	No	Runtime	Function/Block
<code>const</code>	No	Compile-time	Module

3.2.5 Type Annotations

You can explicitly annotate types:

```
let x: i32 = 42
var count: u32 = 0
```

When the type is obvious from context, annotations are optional:

```
let x = 42 // Inferred as i32
let y = 42u32 // Type suffix makes it u32
```

The compiler performs type inference within function bodies. Module-level constants typically require explicit types.

3.3 Primitive Types

Novus provides several categories of primitive types.

3.3.1 Integer Types

Novus supports signed and unsigned integers in multiple sizes:

i8 Signed 8-bit integer (−128 to 127)

i16 Signed 16-bit integer (−32,768 to 32,767)

i32 Signed 32-bit integer (−2,147,483,648 to 2,147,483,647)

i64 Signed 64-bit integer (wide range, requires two 32-bit registers on 68k)

u8 Unsigned 8-bit integer (0 to 255)

u16 Unsigned 16-bit integer (0 to 65,535)

u32 Unsigned 32-bit integer (0 to 4,294,967,295)

u64 Unsigned 64-bit integer (0 to 18,446,744,073,709,551,615)

The default integer type is **i32**. When you write a bare integer literal like **42**, it defaults to **i32** unless context requires otherwise.

3.3.2 Floating-Point Types

f32 32-bit IEEE 754 single-precision float

f64 64-bit IEEE 754 double-precision float

On 68k systems without an FPU, floating-point operations use software emulation—slow but accurate. For performance-critical math on non-FPU systems, prefer fixed-point types.

3.3.3 Fixed-Point Types

Novus provides fixed-point arithmetic for efficient fractional math on integer hardware:

fixed16 16-bit fixed-point (8.8 format: 8 integer bits, 8 fractional bits)

fixed32 32-bit fixed-point (16.16 format: 16 integer bits, 16 fractional bits)

Fixed-point types use native 68k integer instructions, making them dramatically faster than soft-float on systems without an FPU.

3.3.4 Boolean Type

The **bool** type represents truth values:

```
let flag: bool = true
let done: bool = false
```

Booleans result from comparison and logical operations. Unlike C, integers and pointers do not implicitly convert to booleans—you must use explicit comparisons.

3.4 Literals

3.4.1 Integer Literals

Integer literals support multiple bases and optional type suffixes.

Decimal

```
let x = 42
let y = 1_000_000 // Underscores improve readability
```

Underscores are ignored; they exist solely to make large numbers easier to read.

Hexadecimal

Novus supports both C-style and Amiga-style hexadecimal:

```
let color1 = 0xFF00AA // C-style
let color2 = $FF00AA  // Amiga-style (same value)
```

Both forms are equivalent. Use whichever feels natural—Amiga developers often prefer \$, while those from C backgrounds prefer 0x.

Binary

Binary literals use the % prefix:

```
let mask = %1010 // Binary literal (value: 10)
let flags = %11110000
```

Binary literals are useful for bit patterns and masks when working with hardware registers.

Type Suffixes

Append a type suffix to specify the literal's type explicitly:

```
let a = 42u32 // u32
let b = 42i8  // i8
let c = 100u16 // u16
```

Type suffixes work with all bases:

```
let hex_byte = $FFu8
let binary_word = %1010u16
let decimal_long = 1000u32
```

Without a suffix, integer literals default to i32.

3.4.2 Floating-Point Literals

Floating-point literals require a decimal point:

```
let pi = 3.14
let e = 2.71828f32 // f32 with explicit suffix
```

Without a suffix, floating-point literals default to f64.

3.4.3 String Literals

String literals use double quotes and support escape sequences:

```
let greeting = "Hello, Amiga!"
let multiline = "Line 1\nLine 2\nLine 3"
let path = "SYS:Utilities\\" // Escaped backslash
```

Supported escape sequences include:

`\n` Newline (line feed)

`\r` Carriage return

`\t` Horizontal tab

`\0` Null byte

`\\` Backslash

`\"` Double quote

`\'` Single quote

`\xNN` Hexadecimal byte (e.g., `\x1B` for ESC)

3.4.4 F-Strings (Formatted Strings)

F-strings embed expressions directly in string literals:

```
let x = 42
let message = f"The answer is {x}"
// Results in: "The answer is 42"
```

Expressions inside `{...}` are evaluated and converted to strings. F-strings compile to efficient string concatenation or formatting calls.

3.4.5 Character Literals

Character literals use single quotes:

```
let ch: u8 = 'A'
let newline: u8 = '\n'
let escape: u8 = '\x1B' // ESC character
```

Character literals produce values of type `u8`. Novus does not have a separate `char` type—characters are simply 8-bit unsigned integers.

3.5 Operators

3.5.1 Arithmetic Operators

Standard arithmetic works on integer and floating-point types:

Operator	Meaning
<code>+</code>	Addition
<code>-</code>	Subtraction
<code>*</code>	Multiplication
<code>/</code>	Division
<code>%</code>	Modulus (remainder)

Division by zero is a runtime error in debug builds and undefined behavior in release builds. Always validate denominators when accepting untrusted input.

```
let sum = 10 + 20      // 30
let product = 5 * 6    // 30
let quotient = 20 / 5  // 4
let remainder = 17 % 5 // 2
```

3.5.2 Comparison Operators

Comparisons produce `bool` values:

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code><</code>	Less than
<code>></code>	Greater than
<code><=</code>	Less than or equal to
<code>>=</code>	Greater than or equal to

```
let a = 10
let b = 20
let equal = a == b      // false
let less = a < b        // true
let greater_eq = a >= b // false
```

3.5.3 Logical Operators

Logical operators combine boolean values:

Operator	Meaning
<code>&&</code>	Logical AND (short-circuiting)
<code> </code>	Logical OR (short-circuiting)
<code>!</code>	Logical NOT

```
let x = true
let y = false
let and_result = x && y // false
let or_result = x || y  // true
let not_result = !x     // false
```

Both `&&` and `||` short-circuit: the right-hand side is not evaluated if the result is determined by the left-hand side alone.

3.5.4 Bitwise Operators

Bitwise operators manipulate individual bits:

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT (one's complement)
«	Left shift
»	Right shift (arithmetic for signed, logical for unsigned)

```

let a = %1111 // 15
let b = %1010 // 10

let and_bits = a & b // %1010 (10)
let or_bits = a | b // %1111 (15)
let xor_bits = a ^ b // %0101 (5)
let not_bits = ~a // %0000 (-16 in two's complement)

let shifted_left = 1 << 3 // 8
let shifted_right = 16 >> 2 // 4

```

Bitwise operations compile to single 68k instructions and execute in one or two cycles—essential for custom chip programming.

3.5.5 Compound Assignment Operators

Compound assignment combines an operation with assignment:

```

var x = 10
x += 5 // x = x + 5
x -= 3 // x = x - 3
x *= 2 // x = x * 2
x /= 4 // x = x / 4
x %= 3 // x = x % 3

x &= $FF // x = x & $FF
x |= $80 // x = x | $80
x ^= $AA // x = x ^ $AA
x <<= 2 // x = x << 2
x >>= 1 // x = x >> 1

```

All arithmetic and bitwise operators have corresponding compound forms.

3.5.6 Increment and Decrement Operators

Novus supports both prefix and postfix increment/decrement:

```

var x = 5

++x // Pre-increment: x becomes 6, evaluates to 6
--x // Pre-decrement: x becomes 5, evaluates to 5

let a = x++ // Post-increment: a = 5, then x becomes 6
let b = x-- // Post-decrement: b = 6, then x becomes 5

```

Prefix operators modify the variable and return the new value. Postfix operators return the original value and then modify the variable.

3.5.7 Range Operators

Range operators create range values for iteration:

start..end**** Exclusive range (**start** to **end** - 1)

start..=end**** Inclusive range (**start** to **end**)

```
for i in 0..10 {  
    // i goes from 0 to 9  
}  
  
for j in 0..=10 {  
    // j goes from 0 to 10 (inclusive)  
}
```

Ranges are covered in detail in Chapter 5: Control Flow.

3.6 Type Conversions

3.6.1 C-Style Casts

Use C-style cast syntax to convert between numeric types:

```
let x: u32 = 42  
let y: i32 = (i32)x  
let z: *u8 = (*u8)y
```

Casts can chain:

```
let w: i64 = (i64)(i32)(u32)x
```

Casts are explicit and potentially unsafe—truncation, sign changes, and pointer conversions are your responsibility. The compiler trusts that you know what you’re doing.

3.6.2 The **as** Keyword

The **as** keyword provides an alternative syntax for type assertions:

```
let x = 42 as u32
```

Both cast styles are valid; choose whichever feels more natural.

3.7 References and Pointers

Novus distinguishes between references (non-null pointers checked at compile time) and raw pointers (nullable, unchecked).

3.7.1 Immutable References

Create a reference with the **&** operator:

```
var x = 10  
let ref_x: &i32 = &x
```

The type is `&i32`—a reference to an `i32`. References are guaranteed non-null and automatically dereference in most contexts.

3.7.2 Mutable References

For references that allow modification, use `&var`:

```
var x = 10
let ref_x: &var i32 = &var x
*ref_x = 20 // OK - modifies x
```

The type is `&var i32`. The `&var` syntax appears both in the type annotation and when creating the reference.

Note that Novus uses `var` for mutable, not `mut` as in some other languages.

3.7.3 Raw Pointers

Raw pointers use the `*T` syntax:

```
let ptr: *u8 = (*u8)0xDFF000 // Custom chip register
```

Pointers can be null and require explicit null checks:

```
if ptr == null {
    return // Handle null case
}
```

Raw pointers are essential for FFI and hardware access, but prefer references when possible.

3.7.4 Dereferencing

Use `*` to dereference a pointer or reference:

```
let x = 10
let ref_x = &x
let value = *ref_x // 10
```

For pointers, dereferencing an invalid or null pointer is undefined behavior.

3.8 Arrays

3.8.1 Fixed-Size Arrays

Arrays have a fixed size known at compile time:

```
let numbers: [i32; 3] = [10, 20, 30]
```

The type `[i32; 3]` means “array of 3 `i32` values.” The size is part of the type.

3.8.2 Array Literals

Create arrays with bracket syntax:

```
let primes = [2, 3, 5, 7, 11]
```

The compiler infers the size from the number of elements.

3.8.3 Repeat Syntax

Initialize arrays with a repeated value:

```
let zeros = [0; 100] // 100 zeros
let buffer: [u8; 1024] = [0; 1024]
```

The syntax `[value; count]` creates an array of `count` elements, each initialized to `value`.

3.8.4 Indexing

Access array elements with bracket indexing:

```
let numbers = [10, 20, 30]
let first = numbers[0u32] // 10
let second = numbers[1u32] // 20
```

Indices must be unsigned integers. In debug builds, out-of-bounds access triggers a panic. In release builds, bounds checking may be elided for performance.

3.9 Tuples

3.9.1 Tuple Types

Tuples group multiple values of potentially different types:

```
let point: (i32, i32) = (100, 200)
let rgb: (u8, u8, u8) = (255, 128, 64)
```

The type `(i32, i32)` means “a tuple of two `i32` values.”

3.9.2 Tuple Literals

Create tuples with parenthesized, comma-separated values:

```
let color = (192, 96, 32)
```

3.9.3 Destructuring

Extract tuple elements with destructuring:

```
let (r, g, b) = get_rgb()
```

This binds `r`, `g`, and `b` to the tuple’s three elements. Destructuring works in `let` and `var` bindings.

3.9.4 Returning Multiple Values

Tuples enable returning multiple values from functions:

```
fn get_rgb() -> (i32, i32, i32) {
    return (255, 128, 64)
}
```

```
}
```

3.10 Expressions and Statements

In Novus, most constructs are expressions—they produce values.

3.10.1 Expression vs Statement

An expression evaluates to a value:

```
let x = 2 + 2 // Expression: 4
```

A statement performs an action without producing a value:

```
x = 10 // Assignment statement
```

However, even assignments are expressions in Novus, allowing chained assignment:

```
var a = 0
var b = 0
a = b = 5 // Both a and b become 5
```

3.10.2 Block Expressions

Blocks (sequences of statements enclosed in braces) can be expressions if they end with an expression:

```
let x = {
    let a = 10
    let b = 20
    a + b // No semicolon - this is the block's value
}
// x = 30
```

If a block ends with a semicolon, it produces no value (unit type).

3.11 Summary

This chapter covered Novus’s fundamental syntax:

- **Variables:** `var` for mutable, `let` for immutable, `const` for compile-time constants
- **Types:** Integers (`i8–i64`, `u8–u64`), floats (`f32`, `f64`), fixed-point (`fixed16`, `fixed32`), and `bool`
- **Literals:** Decimal, hex (`0x/$`), binary (`%`), strings, f-strings, and characters
- **Operators:** Arithmetic, comparison, logical, bitwise, and compound assignment
- **References:** `&T` (immutable), `&var T` (mutable), `*T` (raw pointers)
- **Arrays:** Fixed-size with type `[T; N]`, indexed with unsigned integers
- **Tuples:** Group values with `(T1, T2, ...)`, destructure with `let (a, b) = ...`

With these building blocks, you can write meaningful programs. The next chapter explores the type system in depth, covering structs, enums, and pattern matching.